

5월 21일 수업용 메모 요약본

🎓 [Day 8] 리눅스 셸(Shell) 프로그래밍 & Git 버전 관리 종합 요약본

💡 **Notion 업로드 팁:** 본 문서는 노션(Notion)의 마크다운 변환에 최적화되어 있습니다. 전체 선택(`Ctrl + A` → `Ctrl + C`) 후 노션 빈 페이지에 붙여넣기(`Ctrl + V`) 하시면 깔끔한 토글(Toggle), 콜아웃(Callout), 테이블(Table) 형태로 자동 렌더링됩니다.

📌 1교시. 셸(Shell)의 이해와 동작 메커니즘

1) 셸(Shell)의 정의

- **정의:** 사용자가 입력한 명령어(텍스트)를 받아들이고 분석하여 운영체제의 핵심인 커널(Kernel)에 전달하고, 그 실행 결과를 다시 사용자 화면에 보여주는 명령어 번역기(Interpreter)입니다.
- **핵심 비역:** "커널이 컴퓨터의 뇌라면, 셸은 사용자와 대화하는 통역사이다."

2) 셸의 종류 및 특징

- **sh (Bourne Shell):** 유닉스 시스템의 최초 기본 셸로, 매우 가볍지만 기능이 제한적입니다.
- **bash (Bourne Again Shell):** 현재 대부분의 리눅스 배포판(Ubuntu 등)의 표준 기본 셸입니다. 강력한 명령어 히스토리 및 자동 완성 기능을 제공합니다.
- **zsh (Z Shell):** 맥OS(macOS)의 기본 셸로, 강력한 플러그인과 테마 시스템을 제공하여 개발자들 사이에서 인기가 높습니다.
- **dash (Debian Almquist Shell):** 시스템 부팅 및 단순 스크립트 실행의 속도를 극대화하기 위해 우분투에서 백그라운드로 사용하는 매우 가벼운 셸입니다.

📌 2교시. `$PATH` 환경변수와 명령어 탐색의 비밀

1) 왜 내 눈앞에 있는 파일(`hello.sh`)을 이름만 치면 실행하지 못할까?

```
kopo20@Ubuntu:~/workspaces/day8$ ls
hello.sh
kopo20@Ubuntu:~/workspaces/day8$ hello.sh
hello.sh: command not found <-- 🚫 왜 못 찾을까?
```

- **원인:** 리눅스 셸은 명령어를 입력받으면 현재 폴더를 검색하는 것이 아니라, 시스템 전역에 설정된 `$PATH` 환경변수 목록 속의 디렉토리들만 뒤지기 때문입니다.

2) 내 컴퓨터의 `$PATH` 구조

```
kopo20@Ubuntu:~/workspaces/day8$ env | grep PATH
PATH=/home/kopo20/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

- 콜론(`:`)으로 구분된 이 경로 지도 위에 현재 작업 공간인 `/home/kopo20/workspaces/day8` 이 나 현재 디렉토리를 뜻하는 기호 `.` 이 없기 때문에 검색에 실패합니다.

3) 💡 보안을 위해 현재 폴더(`.`)를 기본 탐색에서 차단한 철학

- 만약 현재 폴더(`.`)가 기본 탐색 경로에 있다면, 악의적인 사용자가 공용 폴더에 악성 실행 파일을 생성한 뒤 이름을 많이 쓰는 명령어인 `ls` 로 위장해 둘 수 있습니다.
- 관리자(root)가 해당 폴더에 진입하여 무심코 `ls` 를 치는 순간 악성 프로그램이 강제 구동 되는 경로 하이재킹(Path Hijacking)이 발생합니다.
- 리눅스는 이를 예방하기 위해 반드시 실행 위치를 명시하는 보안 설계를 채택했습니다.

📌 3교시. 셸 스크립트 실행하는 3가지 방식의 차이점

실행 방식	명령어 형태	실행 권한(x) 필요 여부	동작 메커니즘
① 인터프리터 직접 지정	<code>bash hello.sh</code>	불필요 (r 권한만 있으면 됨)	이미 실행 권한이 공인된 시스템의 <code>/bin/bash</code> 엔진을 먼저 구동하고, <code>hello.sh</code> 파일은 단순한 원고지 대본처럼 읽어서 번역만 수행함.
② 상대 경로 지정 (권장)	<code>./hello.sh</code>	필수 (<code>chmod +x</code> 필요)	<code>\$PATH</code> 를 경유하지 않고, 현재 폴더(<code>.</code>)의 하위 슬래

			시(/) 주소에 위치한 특정 파일을 콕 집어 운영체제에 직접 단독 프로세스 가동을 요청함.
③ 절대 경로 지정	/home/kopo20/.../hello.sh	필수 (<code>chmod +x</code> 필요)	최상위 뿌리 디렉토리(/) 부터 타깃 파일이 보관된 전체 주소 경로를 적어 지시하는 방식. 예약 스케줄러 (<code>crontab</code>) 등록 시 정석 방식.

파일 실행 날개 달아주기: `chmod +x`

- 텍스트 에디터로 파일 생성 시 기본 상태: `rw-rw-r--` (읽기/쓰기만 가능)
- 명령 실행: `chmod +x hello.sh`
- 실행 후 상태: `rxwxrwxr-x*` (초록색 활성화 및 파일명 끝에 아스터리스크 표시 등록 ➡ 운영체제가 직접 구동할 수 있는 파일임을 입증)

4교시. Git 버전 관리 및 로컬 저장소 구축

1) Git의 3대 핵심 영역 (Life Cycle)

```

[ Working Directory ] —(git add)—> [ Staging Area ] —(git commit)—> [ .git Directory ]
(실제 작업 공간)                (제출 대기 스테이지)                (로컬 저장소/스냅샷)

```

- **Working Directory:** 사용자가 직접 코드를 짜고 파일을 수정하는 리눅스 실제 폴더 공간.
- **Staging Area (Index):** 이번 버전에 포함할 변경 파일들만 임시로 올려놓고 조율하는 무대.
- **.git Directory (Repository):** 커밋이 영구 박제되어 타임머신 기록을 보관하는 로컬 데이터베이스 공간.

2) 로컬 저장소 구축 실무 명령어

```
git init # 현재 작업 폴더를 Git 저장소로 초기화 (.git 파일 생성)
git config --global user.name "ID" # 커밋에 찍힐 개발자 이름 전역 설정
git config --global user.email "EM" # 커밋에 찍힐 개발자 이메일 전역 설정

git add . # 변경된 모든 사항을 Staging Area로 일괄 업로드
git commit -m "feat: first commit" # 로컬 저장소에 영구 버전 스냅샷 저장
git log --oneline --graph --all # 가독성 좋은 한 줄 그래프 형태로 버전 이력 확인
```

5교시. GitHub 원격 저장소 연동 및 협업

1) 로컬 저장소와 원격 저장소 동기화

```
# 원격 저장소 실제 주소를 'origin'이라는 단축 별명으로 로컬에 등록
git remote add origin [https://github.com/사용자ID/리포지토리이름.git] (https://github.com/사용자ID/리포지토리이름.git)

# 현재 등록된 원격지 주소와 권한 목록 상세 스캔
git remote -v

# 내 로컬 master 브랜치의 기록을 원격 저장소 origin으로 최초 밀어 올림
# (-u 옵션은 최초 1회만 사용하며, 이후에는 단순 'git push'만으로 업로드 가능)
git push -u origin master
```

2) 외부 프로젝트 내려받기 및 병합

```
# 다른 PC 환경에서 깃허브 원격 코드를 통째로 로컬로 복제 (최초 1회만 사용)
git clone [https://github.com/사용자ID/리포지토리이름.git] (https://github.com/사용자ID/리포지토리이름.git)

# 원격 저장소의 최신 코드 변경 이력을 감지하여 내 로컬로 병합
git pull origin master
```

6교시. Git 브랜치 생성 및 병합 (Branch & Merge)

1) 브랜치(Branch) 핵심 정의

- **개념:** 기존 안정 버전 코드(예: `master`)의 독립된 카피본(평행우주)을 생성하여, 기존 실행 중인 라이브 서버에 영향을 주지 않고 신기능을 추가하거나 독립 실습을 진행할 수 있게 만드는 논리적 분기 공간입니다.

- **HEAD**: 현재 내가 머물며 편집하고 있는 브랜치의 최신 지점을 가리키는 나침반 포인터입니다.

2) 브랜치 생성 및 제어 흐름

```
git branch                # 로컬 브랜치 목록 스캔 (* 표시가 현재 위치)
git branch feature/login  # 'feature/login' 이라는 신규 가지 생성
git checkout feature/login # 해당 브랜치로 전환 (최근에는 'git switch' 명령어 권장)
git checkout -b feature/login # [꿀팁] 브랜치 생성과 전환을 동시에 원클릭 수행
```

3) 브랜치 병합(Merge) 표준 4단계 시퀀스

가지 브랜치에서 기능 검증이 끝나면 반드시 기준 브랜치로 복귀하여 코드를 병합해야 합니다.

```
[ 1단계: 기준 브랜치 이동 ] - [ 2단계: 최신 업데이트 ] -> [ 3단계: 대상 흡수 병합 ] -> [ 4단계: 잔재 가지 청소 ]
      git checkout master          git pull origin          git merge <대상>
git branch -d
```

```
# Step 1. 수용자가 되는 기준 메인 브랜치로 복귀
git checkout master

# Step 2. 병합 전 원격 저장소의 최신 변경본 일치화
git pull origin master

# Step 3. 가지치기했던 브랜치를 내 품으로 흡수 병합
git merge feature/login

# Step 4. 병합 완수 후 더 이상 쓰지 않는 로컬 가지 안전 제거
git branch -d feature/login
```

4) 병합의 2가지 핵심 알고리즘 작동 원리

- ① **Fast-Forward Merge (속성 병합):**
 - **원인:** 마스터에서 브랜치를 분할한 뒤, 마스터에는 새로운 커밋이 **전혀 없었고**, 가지 브랜치에만 커밋이 쏙 쌓였을 때 발생합니다.
 - **방식:** 복잡하게 코드를 대조하지 않고, **master** 포인터 위치를 가지 브랜치의 최신 커밋 위치로 앞으로 속 옮기기만 하고 한 줄로 합쳐 병합을 빠르게 종료합니다.

- ② 3-Way Merge (3방향 병합):

- **원인:** 마스터에서 분기한 후, 가지 브랜치에서도 개발이 깊어지는 도중 **마스터 브랜치에서도 다른 패치나 팀원 커밋이 동시에 독립 발생했을 때**입니다. (서로 다른 두 갈래 줄기 형성)
- **방식:** 공통의 뿌리(Base) 지점과 양 갈래의 최신 지점 총 3곳을 입체적으로 교차 분석하여 변경 영역을 엮어주고, 새로운 "병합 커밋(Merge Commit)"을 자동으로 개설합니다.

⚡ 7교시. NVIDIA AI 실무자를 위한 Git 시너지 보안 전략

1) **.gitignore** 설정 (대규모 리소스 노출 차단)

- **원인:** AI 딥러닝 프로젝트 시 수십 GB에 달하는 데이터셋 파일이나 체크포인트 가중치 모델 파일(**.pt**, **.pth**, **.onnx**)을 무단으로 푸시하게 되면 깃허브 단일 파일 업로드 한계 용량 (100MB)을 초과하여 푸시 전체가 에러로 먹통이 됩니다.
- **예방책:** 최상위 경로에 **.gitignore** 파일을 개설하여 아래 항목들을 무조건 제외합니다.

```
# --- AI Resource Exclusion ---
dataset/
*.pt
*.pth
*.onnx
.venv/
.ipynb_checkpoints/
```

2) GitHub Token(PAT)과 Secret Scanning 원리

- **질문:** 깃허브 토큰(Personal Access Token)도 일반 API Key처럼 노출되면 위험할까요?
- **답변:** 네, **똑같이 매우 치명적입니다**. 하지만 다른 외부 연동 서비스와 달리 깃허브 내부 시스템에는 **Secret Scanning** 엔진이 탑재되어 있습니다.
- **실시간 구조:** 토큰 정보가 포함된 코드가 Public 리포지토리에 푸시되는 순간, 엔진이 소스 코드의 문자열 패스를 감지하여 **0.1초 만에 해당 토큰을 자동으로 강제 즉시 만료(Revoke)** 처리하고 주체 계정 이메일로 비상 통보를 보냅니다. 대형 해킹 사고를 미연에 방어해 주는 강력한 안전망 장치입니다.

🚨 8교시. 실무 및 실습 단골 에러 & 해결책 백과사전

▼ [Toggle] 에러 1: `hello.sh: command not found` (명령어 인식 실패)

- **원인:** 실행하고자 하는 파일이 있는 디렉토리가 시스템 명령어 경로 보관소인 `$PATH` 환경 변수 목록에 기재되어 있지 않기 때문입니다.
- **해결 방법:** 경로를 명시하는 상대 경로 기호 `./` 를 파일명 앞에 강제로 붙여 실행을 명시 선언합니다:

```
./hello.sh
```

▼ [Toggle] 에러 2: `bash: ./var.sh: Permission denied` (권한 부족 거부)

- **원인:** 리눅스는 기본 보안 원칙상 생성되는 텍스트 스크립트 파일에 대해 단독 구동 실행 권한(`x`)을 부여하지 않고 생성하기 때문에 발생합니다.
- **해결 방법:** `chmod +x` 를 통해 실행 권한을 수동 활성화합니다:

```
chmod +x var.sh
./var.sh
```

▼ [Toggle] 에러 3: `chown: invalid user: 'kopo30:kopo30'` (존재하지 않는 사용자)

- **원인:** 깃 전역 닉네임 설정(`git config --global user.name "kopo30"`)을 한 것은 커밋 로그 파일에 적힐 닉네임 문자열을 바꾼 것뿐이지, 실제 리눅스 운영체제 사용자 계정(`/etc/passwd`)을 만든 것이 아니기 때문에 발생합니다.
- **해결 방법:** 운영체제가 실제로 가동 중인 본연의 리눅스 계정명(`kopo20`)으로 소유권 이전 대상을 지정해야 합니다:

```
sudo chown kopo20:kopo20 README.md
```

▼ [Toggle] 에러 4: `bash: :w: command not found` (터미널 쉘 오타 실수)

- **원인:** 읽기 전용 파일을 루트 관리자 권한으로 강제 우회 저장할 때 쓰는 `:w !sudo tee %` 구문은 Vim 에디터 명령 모드 전용 스크립트입니다. 이를 에디터 바깥인 터미널 쉘 화면에 직

접 타이핑하여 발생한 쉘 오해 오류입니다.

- **해결 방법:** 이미 에디터 바깥 터미널 쉘로 빠져나온 상황이라면, 터미널 표준 권한 명령어를 쳐서 해결합니다:

```
sudo chown kopo20:kopo20 README.md
chmod +w README.md
```

▼ [Toggle] 에러 5: VS Code README.md 수정 시 "Permission Denied" (중복 세션 충돌)

- **원인:** 동일한 소스파일을 다중 화면이나 에디터 다중 탭(Tab)으로 중복 열어두었을 때, 프로그램이 소스코드의 동시 수정 고임을 막기 위해 임시 강제 쓰기 락(Lock)을 걸어 읽기 전용 상태로 전환하기 때문에 발생합니다.
- **해결 방법:** 열려 있는 동일한 파일의 다른 중복 탭들을 모두 닫고 세션을 1개로 일치시켜 주면 즉시 락이 해제되어 정상 수정이 가능합니다.

▼ [Toggle] 에러 6: **CONFLICT (content): Merge conflict in <파일명>** (코드 충돌 제어)

- **원인:** 동일한 파일의 동일한 코드 라인을 두 작업자(또는 두 브랜치)가 동시에 제각각 다르게 수정한 뒤 병합을 지시했을 때, 깃이 최후의 우선권을 판단하지 못하고 개발자에게 수동 결정을 요청하며 중단하는 현상입니다.
- **해결 방법:**
 1. 충돌이 선언된 소스파일을 엽니다.
 2. 코드 사이의 경계선 표시(<<<<<< HEAD, =====, >>>>>>)와 내부 소스코드를 비교하여 합의된 최후의 코드로 수동 병합 및 라인을 정리합니다.
 3. 경계선 찌꺼기 텍스트를 깨끗하게 지우고 새로 저장한 뒤 다시 버전 기록 절차를 완료합니다:

```
git add <충돌정리파일명>
git commit -m "Merge branch 'feature/login' (resolved conflict)"
```

▼ [Toggle] 에러 7: **fatal: repository '...' not found** (잘못된 깃 주소 수신)

- **원인:** 브라우저 창에서 특정 브랜치의 뷰화 주소(예: `/tree/master` 혹은 `/tree/master.git` 등이 꼬리에 추가된 주소)를 통째로 복사해 `git clone` 매개 주소로 제공했을 때, 존재하지 않는 꼬리 디렉토리 저장소 파일 구조를 찾으려다 유실 실패하는 현상입니다.
- **해결 방법:** 뒷단의 웹 브라우저 부가 뷰 디렉토리 꼬리 명세를 전부 배제하고, 순수한 **리포지토리 루트 코어 URL** 주소만을 활용해 클론을 지시해야 합니다:

```
# 올바른 최상위 표준 원격지 주소 입력
git clone [https://github.com/Salmonyeonwoo/linux_day8.git](https://github.com/Salmonyeonwoo/linux_day8.git)
```